

2.9 - Arquitetura orientada a eventos

Producers, consumers e brokers

Os elementos da arquitetura de eventos

Quatro peças, um modelo mental novo

- **Producer publica um fato que aconteceu no sistema**
 - Não sabe nem se importa com quem vai consumir
- **Consumer reage ao evento de forma independente**
 - Vários consumers podem reagir ao mesmo fato
- **Broker transporta e desacopla producer de consumer**
 - Kafka, RabbitMQ e SQS são exemplos comuns
- **Evento é registro imutável de um fato passado**
 - Descreve o que ocorreu, nunca uma ordem a cumprir

Um evento é um fato, não uma ordem

JAVASCRIPT

```
// Evento: descreve algo que já aconteceu
const entregaConfirmada = {
  tipo: 'EntregaConfirmada',
  id: 'evt-9f2a',
  ocorridoEm: '2026-05-26T14:32:00Z',
  entregaId: 'ent-7781',
  transportadoraId: 'tr-204'
};

// O producer publica e segue a vida
broker.publicar('entregas', entregaConfirmada);
```

Message queue vs. publish/subscribe

Message queue (1 para 1)

- Cada mensagem vai para um único consumer
- Ideal para distribuir trabalho entre workers
- Processamento de tarefas e filas de jobs
- Garante que só um consome cada item

Publish/subscribe (1 para N)

- Cada evento chega a todos os interessados
- Ideal para reagir a um fato em vários lugares
- Notificação, auditoria e sincronização em paralelo
- Producer ignora quantos consumers existem

Seis coisas em uma mudança de status

- **Uma mudança de status dispara seis reações diferentes**
 - Fazer tudo síncrono acoplaria o fluxo inteiro

LUNALOG

Quando uma entrega muda de status na LunaLog, seis coisas precisam acontecer: notificar o cliente, atualizar o dashboard da transportadora, disparar o billing, atualizar o histórico do motorista, gerar o evento de auditoria e sincronizar com o ERP do cliente. Fazer isso de forma síncrona seria uma cadeia de chamadas acopladas, onde qualquer falha derruba o fluxo todo. Ana propôs publicar a mudança de status como um fato em um broker, e deixar cada interessado reagir no seu tempo. O acoplamento caiu, o fluxo ficou resiliente.

Eventos invertem a direção do acoplamento

- Em chamadas diretas, quem chama conhece quem responde
- Em eventos, o producer ignora quem está ouvindo

A arquitetura de eventos troca acoplamento por autonomia. Adicionar um novo consumer não toca no producer. Em troca, você abre mão da resposta imediata e aceita consistência eventual.

- Use EDA quando vários reagem ao mesmo fato



O modelo de eventos parece elegante e desacoplado, e é, até você colocar em produção. Aí os eventos chegam fora de ordem, se repetem sem aviso e às vezes somem no caminho. Cada um desses problemas tem um custo financeiro concreto. O que acontece quando o mesmo evento é processado duas vezes?

EDA: ordenação, idempotência e rastreabilidade